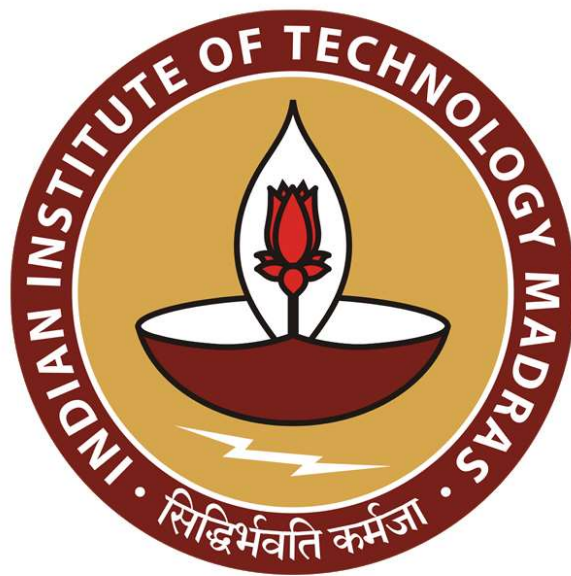




IIT Madras
ONLINE DEGREE

Database Management Systems
Professor. Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
Algorithms and Data Structures/2: Data Structures

Welcome to Module 37 of Database Management Systems course, in the IIT Madras online BSc program.

(Refer Slide Time: 0:38)

The screenshot shows a presentation slide titled "Module Recap" from the IIT Madras online BSc program. The slide is part of Module 37, "Database Management Systems". The slide content includes a list of bullet points: "Need for analyzing the running-time and space requirements of a program", "Asymptotic growth rate or order of the complexity of different algorithms", and "Worst-case, average-case and best-case analysis". A video feed of Professor Partha Pratim Das is visible in the bottom right corner of the slide.

We have been discussing about fundamentals of algorithms and data structure, which you have already done in your algorithm's course, but we are just doing a quick recap of the key ideas and key data structure which will be frequently needed for our design and optimization of the physical storage.

So, we talked about the analysis of an algorithm particularly the execution time and space. We have recapped on the asymptotic growth and looked at some of the, provided a chart of complexity for the common data structures and common sorting algorithms.

(Refer Slide Time: 1:16)

Module Objectives

- Introduction to Data Structures
- Review of linear data structures - array, list, stack, queue
- Review of search - linear and binary

Database Management Systems | Partha Pratim Das | Page 21/22

Module Outline

- Linear data structures - array, list, stack, queue
- Search - linear and binary

Database Management Systems | Partha Pratim Das | Page 21/22

In this module, we will again, talk specifically about data structures, simple data structures. We will review the linear data structures and also the linear and binary search. So, that is what is on the agenda.

(Refer Slide Time: 1:30)

The screenshot shows a presentation slide titled "Data Structure" from IIT Madras. The slide content is as follows:

- **Data structure:** A data structure specifies the way of organizing and storing in-memory data that enables efficient access and modification of the data.
 - Linear Data Structures
 - Non-linear Data Structures
- Most data structure has a container for the data and typical operations that its needs to perform
- For applications relating to data management, the key operations are:
 - Create ✓
 - Insert ✓
 - Delete ✓
 - Find / Search ✓
 - Close ✓
- Efficiency is measured in terms of time and space taken for these operations

So, a data structure, as you know, is it specifies a way of organizing and storing in-memory data that enables efficient access and modification of the data. You are studying database, so you will often be encountering this question that what is the difference between a database and a data structure. Be prepared to address that.

A data structure, which broadly we talk of is necessarily in-memory, that is, it exists as long as the program is in execution, and a data structure has a persistency. There are other factors also, but this is a key difference. The data structure typically can be classified in two ways. One is linear data structure, the other is nonlinear data structures. We will take one by one.

Now, in designer's perspective, what is a data structure? It is a container, it must contain data because it is a data structure, so the primary of it is a container and it has a set of typical operations. The operations that you would like to do efficiently with an efficient container storage. So, this combination of what is the kind of container and what are the type of operations you want to do, tells you specifically about the data structure.

Now, there are plethora of data structures, but we are focusing on the fundamentals ones, particularly relating to data management because, we are doing it as a precursor to the physical storage design. So, in relation to data management, I am immediately not talking about what the container could be it could vary.

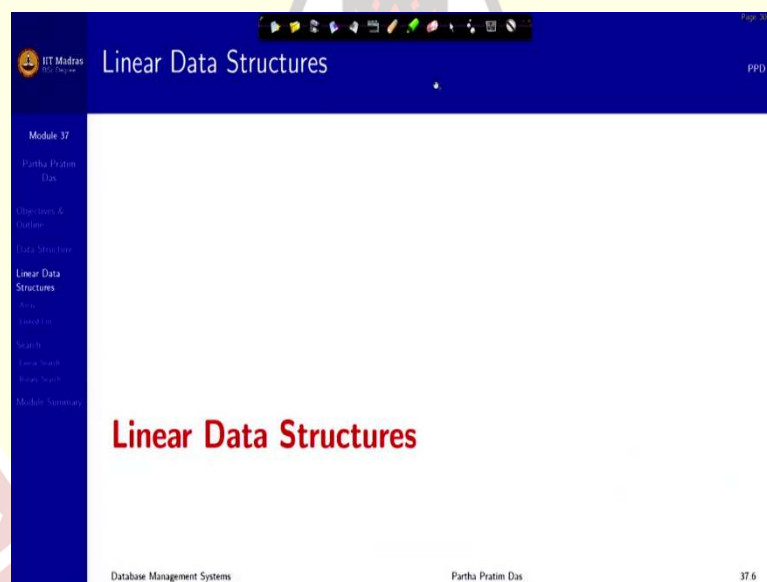
And as we will go finally, to the database physical layer we will see that it will have containers, which are no more in-memory to give you persistency they are outside of the memory system, they are in that permanent storage and so on, but the key operations of interests are these five.

Certainly, you need to start creating it somewhere. There is a create. You would like to add a data insert, delete, remove a data, find a data.

So, we will, we can talk about update, but we will not specifically say that because an update is kind of a search, delete insert like, by this you can actually have an update though you may have direct update procedures also. And finally, you close the data structure wrap up whenever it is not required.

So, when we talk about efficiency, efficient access, what we mean is it is efficient the data structure is efficient in terms of time and space for these operations, efficiency is not absolute. Some data structure is efficient for some operation, some is efficient for some other operation. Some is efficient in space, some may be more efficient in time and so on so forth. These factors will have to be all kept in mind, as we go over this discussion.

(Refer Slide Time: 5:02)



HT Madras
Linear Data Structures

Module 37
Partha Pratim Das
Objectives & Outline
Data Structures
Linear Data Structures
Array
Linked List
Search
Linked Search
Tree Search
Module Summary

- A Linear data structure has data elements arranged in linear or sequential manner such that each member element is connected to its previous and next element.
- Since data elements are sequentially connected, each element is traversable through a single run.
- Examples of linear data structures are Array, Linked List, Queue, Stack, etc.

So, first we talk about linear data structures. A linear data structure has data elements arranged in a linear sequential order. And in some way, there is a way to know if you know a particular data element, then there is a way to know the next data element. So, if you look at the computer organization, there are two ways. One is purely sequential, that is like memory where you have a one after the other, you have the data and they have addresses.

When is the, where is the next data element? Where this data element ends. They are necessarily consecutive, so we can also go to any arbitrary data element by looking at the by referring the index. Because I know, if I know where this starts by knowing it is the fourth element, I will know what this offset is, and I can directly go there. So, this is indexed, this is random access.

What I am trying to say is before coming to data structure, I am just trying to see what does the computer organization give us because our all our data structure will have to be based on it. The other one that we have is referential, which is, I have a location, memory location, which has an address of the memory location. So, if I know the address here, I can go to this element.

So, this is by, you can say call it a reference, you can call it a pointer, but it is referential, that is, I can use the memory addresses to go from one data to the other. So, here I have to know with every data element, as to what is the address where the next data element will be available. That is a basic concept of the referential mechanism. These are the two fundamental mechanisms available by computer organization, so all your data structures are based on these two concepts.

(Refer Slide Time: 7:51)

Linear Data Structures

- A Linear data structure has data elements arranged in linear or sequential manner such that each member element is connected to its previous and next element.
- Since data elements are sequentially connected, each element is traversable through a single run.
- Examples of linear data structures are Array, Linked List, Queue, Stack, etc.

A	B	C	D	E	F	G	H
---	---	---	---	---	---	---	---

So, based on that we have arrays, which is indexed, sequential, we have linked list which is referential, and using those we could build, queue or stack, which could either use indexed, sequential or it could use the referential structure. And as you all know its consecutive locations, I will.

(Refer Slide Time: 8:15)

Linear Data Structures (2)

Different examples of linear data structure:

- **Array:** The data elements are stored at contiguous locations in memory.
- **Linked List:** The data elements are not required to be stored at contiguous locations in memory. Rather each element stores a link (a pointer to a reference) to the location of the next element.
- **Queue:** It is a FIFO (First In First Out) data structure. The element that has been inserted first in the queue would be removed first. Thus, insert and removal of the elements in this take place in the same order.
- **Stack:** It is a LIFO (Last In First Out) data structure. The element that has been inserted last in the stack would be removed first. Thus, insert and removal of the elements in this take place in the reverse order.

So, stored in contiguous location, it can be randomly accessed and so on. In linked list the data elements are not stored in contiguous location rather each element has a link has a reference has a pointer to the location in that. Now, when we talk about other data structures like queue and stack, we will first talk about operations. So, what are the operations here?

Let us say, what is the operation in an array? The operation in an array, I can create an array. I can insert an element in there, I can delete an element from there, I can search for an element

in the array. In the linked list, I can create a linked list and interestingly I can keep on creating new data nodes in the linked list, which is an additional operation which array does not have, but in addition, I can insert a value, I can delete, I can search all these I can do. But so in array and linked list, we have the container and the operations both defined together.

In case of a queue or stack, we do not talk about the container primarily, we primarily talk of the operations. So in queue, what we say, it is a first in first out. So, a queue, if I remove an element from the queue, then it will always be the oldest element residing in the queue. That is the insertion and deletions take place in the same order. So, it is a first in first out.

Whereas, in a stack, when I remove an element, I will always get the last inserted element, last in, first out. It is not typical for a queue or a stack to have any other kind of say find operation. Finding an element in stack is not what you do. Finding an element in queue we do not know.

Instead you have in queue you say what is the front element, that if I remove, what is the next person who is going to get to the counter. In stack you say what is the topmost element, so it is operations are a little different connotation. Naturally will have a create, you will have closure, but what is significant to see that the basic definition of a queue or a stack does not depend on the container, it is only on the operations.

Now, a queue can be on an array or a linked list, it could be circular in an array and so on so forth, so container could vary, container structure could vary, but the operations will have to remain same that is what defines a data structure. Similarly for the stack, kind of stack on an array stack on a linked list. You know that, you know all this. I am just trying to summarize what you know.

(Refer Slide Time: 11:21)

Linear Data Structures (3): Array

- The elements are stored in contiguous memory locations.

- Simple access using indices. For example, let the array name be arr, we can access the element at position 5 as arr[5].
- Array allows random access using its index which is fast (cost of $O(1)$). Useful for operations like sorting, searching.

So, array is indexed, you already know that on one side you have the array indices, on the other side, you have the physical memory address and inside you have the containers. So, you can, it is computer organization wise it is implemented in a way so that the accessing every element is very fast that is order one, it is a constant time because you know this arithmetically, you can compute the access and get this address is just one memory, very simple.

(Refer Slide Time: 11:58)

Linear Data Structures (4): Array

- Have fixed sizes, not flexible.** Since we do not know the number of elements to be stored in runtime. If we create it too large then it can be a waste of memory, if we create it too small then some elements may not be accommodated in the array.
 - For example, suppose we create an array to store 8 elements. However, during execution of the program only 5 elements are available, which results in wastage of memory space.

Database Management Systems

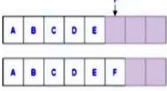
Arrays have fixed size they are not flexible. You may have heard of some variable size array and all that, but those are implementation details, but conceptual an array has a fixed size. So, array may have, if I decide to use the space from the left end, the 0 index end then array will

have some unused space at the towards the end. And as long as there is space, I can use that array.

(Refer Slide Time: 12:29)

Linear Data Structures (5): Array

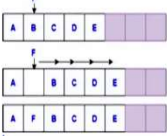
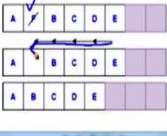
Module 37
Partha Pratim Das

- Insertion and removal of elements from an array are costlier since the memory locations have to be consecutive.
 - Insertion or removal of an element from the end of an array is easy.
 - ▷ Insert at end:

 - ▷ Remove from end:


Database Management Systems Partha Pratim Das 7.11

Linear Data Structures (6): Array

Module 37
Partha Pratim Das

- Insertion and removal of elements from an array are costlier since the memory locations have to be consecutive.
 - Insert and remove elements at any arbitrary position is costly (cost is $O(n)$)
 - ▷ Insert at any arbitrary position:

 - ▷ Remove from any arbitrary position:


Partha Pratim Das

So, if I want to do insertion or deletion, it is very easy, if I am inserting at the end, because there is something some empty space, I just put the element there or if I am removing from the end, it is the last element so I remove, so that is constant time. But if I have to insert at an arbitrary position, as in here, then I have a problem, because I have to create space for the element I want to insert, so I have to move all of these, right. So, I have to copy this here, copy this here, copy this here, and only then I have this the space for F.

So, the worst case would be, when I want to insert, when I want to insert in the front have to move all the elements. So naturally, insertion in an array becomes ordered n , in general,

similarly removal will be order n as well because after removing this you have to move these elements left towards back so that you satisfy the assumption that you have elements packed from index 0 to a certain point. So, array is good for access, but not so good for insert and delete.

(Refer Slide Time: 14:01)

Linear Data Structures (7): Linked List

- Elements are not required to be stored at contiguous memory locations. A new element can be stored anywhere in the memory where free space is available. Thus, it provides better memory usage than arrays.
- For each new element allocated, a link (a pointer or a reference) is created for the new element using which the element can be added to the linked list.

Each element is stored in a node. A node has two parts:

- **Info**: stores the element.
- **Link**: stores the location of the next node.

• Header is a link to the first node of the linked list.

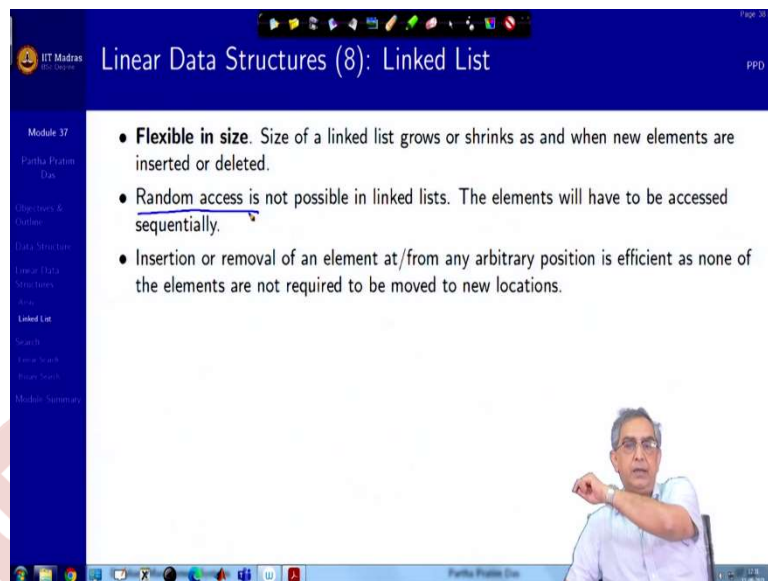
Now, let us move on to a linked list. They are not in contiguous location, so every data has two parts. One is a data and another is a link where the next data happens. And to know where the first data is, you have a special storage called header.

So, in this, naturally, you will have only those many data nodes as there is data. Unlike array where you will have extra. So when do you say is there is nothing more to find, just you put a 0. That is again a system specification comes from computer organization. So, this is your linked list.

(Refer Slide Time: 14:50)

Linear Data Structures (8): Linked List

- **Flexible in size.** Size of a linked list grows or shrinks as and when new elements are inserted or deleted.
- **Random access is not possible in linked lists.** The elements will have to be accessed sequentially.
- **Insertion or removal of an element at/from any arbitrary position is efficient** as none of the elements are not required to be moved to new locations.



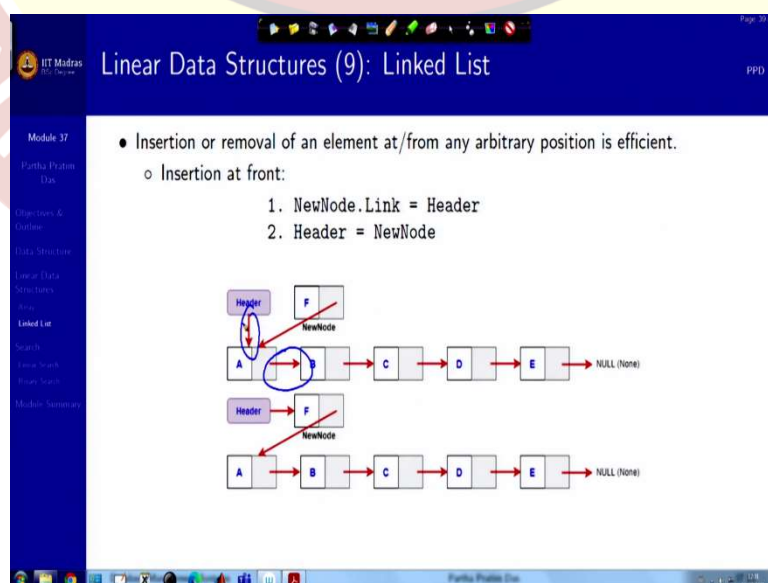
So, it is flexible in size, but the random access that you could have in an array is no more possible because you do not know the address of the third element. The address of the third element is known by the second element's link. The address of the second element is known in the first element's link the address of the first element is known in the header, so, you will have to necessarily start from the header and go there.

So, immediately you can see that access turns out to be order n , because, you could be looking for accessing the last element. So, you have to start in the header pass through the entire list order n expensive, in array it was order 1.

(Refer Slide Time: 15:45)

Linear Data Structures (9): Linked List

- Insertion or removal of an element at/from any arbitrary position is efficient.
 - Insertion at front:
 1. `NewNode.Link = Header`
 2. `Header = NewNode`



Linear Data Structures (9): Linked List

Module 37
Partha Pratim Das

Objectives & Outline
Data Structures
Linear Data Structures
Array
Linked List
Search
Linear Search
Binary Search
Module Summary

- Insertion or removal of an element at/from any arbitrary position is efficient.
 - Insertion at front:
 - `NewNode.Link = Header`
 - `Header = NewNode`

So, if you look at insertion of an element in the beginning, it is very easy. You have a node just all that you need to do is to adjust these pointers. So, A was pointing what is it, A was pointing to B and header was pointing to A instead you wanted at the front, so this one you want at the front, so what do you have to make, you have to make header point to this, and F points to A. So, you can make F points to A and then just change the header pointer. Very simple constant time.

(Refer Slide Time: 16:29)

Linear Data Structures (10): Linked List

Module 37
Partha Pratim Das

Objectives & Outline
Data Structures
Linear Data Structures
Array
Linked List
Search
Linear Search
Binary Search
Module Summary

- Insertion or removal of an element at/from any arbitrary position is efficient.
 - Remove from front:
 - `Temp = Header`
 - `Header = Header.Link`
 - `Delete(Temp)`

Database Management Systems Partha Pratim Das 37.16

Linear Data Structures (11): Linked List

Module 37
Partha Pratim Das
Objectives & Outline
Data Structures
Linear Data Structures
Array
Linked List
Search
Linear Search
Binary Search
Hashing Summary

- Insertion or removal of an element at/from any arbitrary position is efficient.
 - Insertion at end:
 - `Node.Link = NewNode`

Page 41/41

PPD

Similar thing you can think of in terms of the removal, if you are removing the headlock, constant time again. Now what if you want to insert at an arbitrary position. So, the first assumption would be, we have already seen that the access is order n . So, we cannot mix up with that. What we will say is, if I know where to insert, if I know where to insert, I will assume that I know where to insert then it is very easy to insert. If I know to insert at the end, then all that I need to know is this node.

If I know this node, then insertion is just putting this link. But if I do not, I will need linear time to come to this node, the last node. So, assuming that I know the last node, it is a constant time operation.

(Refer Slide Time: 17:39)

Linear Data Structures (12): Linked List

Module 37
Partha Pratim Das
Objectives & Outline
Data Structures
Linear Data Structures
Array
Linked List
Search
Linear Search
Binary Search
Hashing Summary

- Insertion or removal of an element at/from any arbitrary position is efficient.
 - Remove from end:
 - `Temp = Node.Link`
 - `Node.Link = NULL`
 - `Delete(Temp)`

Page 42/42

PPD

Similarly, for removing from the end, all that I need to know is, the node before the last node. I cannot remove the last node by itself because it is being referenced from the previous node. So, I need to, if to remove this node, I need to know this node. So, if I assume that I know this node, then it is a constant time. All that I need to do is to set this to NULL and get rid of the last node.

(Refer Slide Time: 18:12)

Linear Data Structures (13): Linked List

- Insertion or removal of an element at/from any arbitrary position is efficient.
 - Insertion at any intermediate position:
 - `NewNode.Link = Node.Link`
 - `Node.Link = NewNode`

The diagram for Slide 13 shows a linked list with nodes A, B, C, D, E. A new node F is being inserted between C and D. The pointer of node C is updated to point to node F, and the pointer of node F is updated to point to node D.

Linear Data Structures (14): Linked List

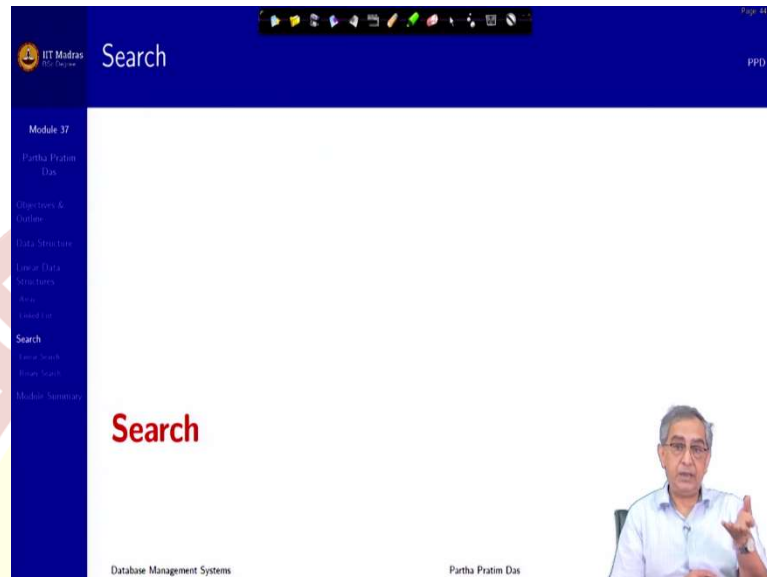
- Insertion or removal of an element at/from any arbitrary position is efficient.
 - Remove from any intermediate position:
 - `Temp = Node.Link`
 - `Node.Link = Node.Link.Link`
 - `Delete(Temp)`

The diagram for Slide 14 shows a linked list with nodes A, B, C, D, E. Node C is being removed. The pointer of node B is updated to point to node D, and node C is marked for deletion.

Similar observations can be made at intermediate position. If I know the position where I want to do the node, then all that it needs, if I have to insert this F all that I need is to adjust this and adjust this. That F has to, the node after which F has to come that its pointer should be set to F and F's pointer should be set to the node it was originally pointing to constant time.

But the key of all of these is the fact that I need to know the node where I am actually doing the insert. Similarly, remove would be also constant time order 1, if I know the node before the node that I want to remove.

(Refer Slide Time: 18:56)



So, we can see that to sum up in terms of array, the access is very efficient order 1, the insert and delete are order n , whereas, in a linked list access is order n but insert delete once you know the position is very efficient it is order 1, so you have a trade-off in that way. Let us leave with it.

So, the most, the other common operation that is left is search, how to find. That is different from access. In access, you have the index and you are finding, so it is positional, but search is by value, so it is associational, associative search, that given the content you find out where does it exist.

(Refer Slide Time: 19:49)

Linear Search

- The algorithm starts with the first element, compares with the given key value and returns yes if they match.
- If it does not match, then it proceeds sequentially comparing each element of the list with the given key until a match is found or the full list is traversed.

Let the given input list be `inputArr = ['a','c','a','d','e','m','i','c','s']` and the search key be 'i'.

	a	c	a	d	e	m	i	c	s
Step 1	a = ?								
Step 2		c = ?							
Step 3			a = ?						
Step 4				d = ?					
Step 5					e = ?				
Step 6						m = ?			
Step 7							i = ?		

Figure: Linear Search Example

So, one way you could do is to do a linear search, that given an element you start and check with every element in an array, you will be able to do that in, and if that element exists, you will be able to say that it exists, if it does not, if you have exhausted the whole array, you will say, it does not exist. So, you can do a linear search, which will obviously be order n .

(Refer Slide Time: 20:26)

Linear Search (2)

Python Code for Linear Search:

```
def linSearch(inputArr, k):  
    for i in range(len(inputArr)):  
        if inputArr[i] == k:  
            return i  
    return -1  
  
inputArr = ['a','c','a','d','e','m','i','c','s']  
k = 'i'  
index = linsearch(inputArr,k)  
if index != -1:  
    print("Element found at "+ index)
```

Binary Search

- The input for the algorithm is a sorted list.
- The algorithm compares the key k with the middle element in the list.
- If the key matches, then it returns the index.
- If the key does not match and is greater than the middle element, then the new list is the list to the right of the middle element.
- If the key does not match and is less than the middle element, then the new list is the list to the left of the middle element.

Let the given input list be `inputArr = ['a','a','c','c','d','e','i','m','s']` and the search key be 'i'.

Step 1: $d = i?$

Step 2: $i = i?$

Figure: Binary Search Example

$O(\log n)$

Binary Search

- The input for the algorithm is a sorted list.
- The algorithm compares the key k with the middle element in the list.
- If the key matches, then it returns the index.
- If the key does not match and is greater than the middle element, then the new list is the list to the right of the middle element.
- If the key does not match and is less than the middle element, then the new list is the list to the left of the middle element.

Let the given input list be `inputArr = ['a','a','c','c','d','e','i','m','s']` and the search key be 'i'.

Step 1: $d = i?$

Step 2: $i = i?$

Figure: Binary Search Example

$\log_2 n \approx \log_2 9$

Now, here is a simple Python code, which lets you do that. The other option that, you know, I am sure you know, is there is a better way to search this. What do you do? Assuming that the elements that you want to store and search can be ordered. I mean, one can be considered to be less than the other, there is some way to compare them, what you do is you sort the elements in certain order increasing or decreasing whatever order in the array.

So, there is a setup time that is required for this, so create of this is going to take more time than just an array because it is just not storing the elements, you have to also sort them so that you can do this kind of a search. So, that setup is going to take you order $n \log n$ time.

But once the elements are sorted, and you are given to search for something, you have a very simple strategy. Is that you look for, instead of starting from the beginning to end, you look for the middle element, and you compare with the middle element. You know they are sorted say,

in this case, they are sorted in increasing order. You are looking for I , so as you compare for the middle element, you see it is d .

Now, what is the assertion that you can make? Is since they are increasingly ordered and i is greater than d , i can never if it at all happens, it can never happen on the left side, so you remove the left side or not physically remove, remove it from the consideration, and therefore, your problem reduces to searching for I in this part of the array, which is half the size, and then you again look at the middle it is even so, you have to look at either the I node or the m node anyway, if you happen to look here, then you will immediately find i and you will say it exist. If you do not find, you will again half because it can either be on the left or be on the right.

Finally, given so, first you will search within array of size n , then in an array of size n by 2, then in an array of size n by 2 square you keep on doing till you will obviously be left with an array of size 1, it either exists that, it either matches that or it does not. If it does not, it does not exist, otherwise, it matches somewhere in between. So, as many times you will do this division and matching is your complexity of operation.

How many times can you divide n by 2 so that it becomes 1? That in fact is a value of $\log n$, incidentally, when I write $\log n$, what I mean is $\log$ to the base 2 n . When I write $\lg$ it means and when I write $\log$, it means the natural logarithm. So, $\lg$ is what is very common. So, how many times can you divide? If I take 16.

I can divide by 2, get 8 first, divide 8 by 2 get 4, second, divide 4 by 2 get two, third, divide 2 by 2 get one, fourth. So, I have got to 1. So, how many steps? Four. What is $\log 16$ base 2 is 4, that is a power that you will have to take of 2 to get to 16. So, which says that the beauty of this search naturally called a binary search is you can find a key if, in or determine that it does not exist in $\log n$ time.

(Refer Slide Time: 24:57)

Binary Search

Python Code for Binary Search:

```
def binSearch(inputArr, k):
    low = 0
    high = len(inputArr) - 1
    mid = 0
    while low <= high:
        mid = (high + low) // 2 # Division(floor)
        if inputArr[mid] < k: # new list is to the right of k
            low = mid + 1
        elif inputArr[mid] > k: # new list is to the left of k
            high = mid - 1
        else: # means k is present at mid
            return mid
    return -1 # The element is not present

inputArr = ['a','a','c','c','d','e','i','m','s']
k = 'i'
index = binSearch(inputArr, k)
if index != -1:
    print("Element found at position "+ str(index+1))
else:
    print("Not found ")
```

Handwritten notes on the first slide: Search, Index

Handwritten notes on the second slide:

	Array		List	
	Ord	Unord	Ord	Unord
Linear	$O(n)$	$O(n)$	$O(n)$	$O(n)$
BS	$O(\log n)$	$\times$	$\times$	$\times$
Ins	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Del	$O(n)$	$O(n)$	$O(n)$	$O(n)$

Handwritten note: LDS

So, these are the two ways to do the basic search mechanism. And there is a Python code. Now, if we put it to context then for search if I have linear, if I do linear. Let me write it more cleanly. So, if I do linear search I can do it in an array, which could be ordered or unordered it does not matter, and in both case, it is order n . If I do a binary search then if it is ordered then I can do in order $\log n$. If it is unordered I cannot do it.

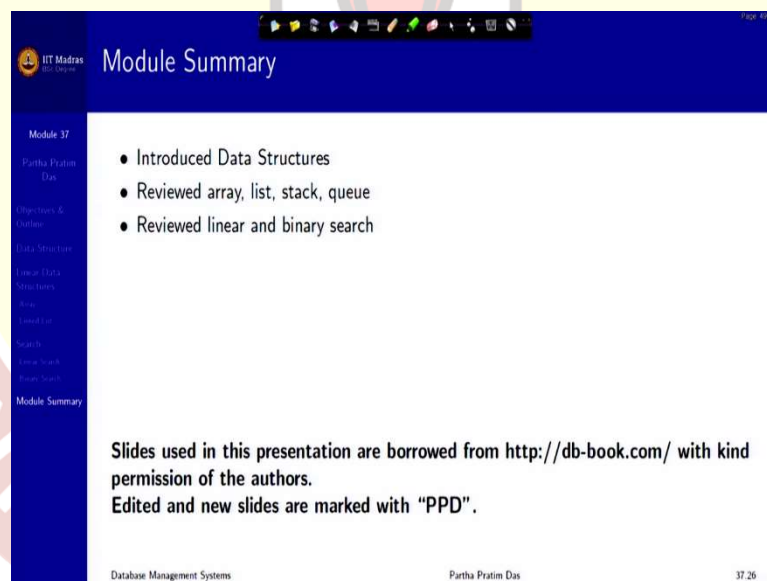
If I have a linked list, it can be ordered, it can be unordered. The linear search will always be order n . Binary search simply cannot be done even if it is ordered because the to get to the middle I need to have an access to the middle, and that access is itself order n .

Insert in an array it is order n , in an unordered array it is order 1. Why is it order 1? Because if the array is not ordered, then I can insert it anywhere. I can insert it at the end. The deletion, the insertion in linked list is order 1, whether it is ordered or unordered. How about deletion?

In an array it is order n , whether it is ordered or it is not ordered, because I have to delete from wherever it is and move the rest, whereas, on linked list it has order 1. So, you can see of my three major operations if one is good in in one data structure like insert or delete is good in linked list, the other the search or find is bad it is order n . And if search is good in a array, the insert delete is bad.

So, my conclusion is that my linear data structure has contradicting performance. It does not give me a complete performance, so that all of insert, delete, search can be done efficiently. Okay, I mean, I am not saying that it has to be order 1 all the time, but at least it has to be order $\log n$. Order n is too expensive to work with. And you should look at the growth chart again to see what is the great difference between order n and order $\log n$.

(Refer Slide Time: 28:54)



So, that is what we have recapture today in I mean, reintroducing the data structure I should say and reviewing array, list, stack, queue and so on. And naturally if we, if you look at the complexity of the stack operations and queue operations, whether you do them with an array or with a linked list, you will see that the basic pop, push, top or add, delete, remove, add, delete, check front item in queue all of them we have to order 1, the constant time.

So, with that, I will close the, we close this module. Thank you for your attention, and we will meet in the next module to talk about the nonlinear data structures.

